

SoNSo Website: Technical Spec (Lite)

Oct 3, 2026 · @Sampath

Plain HTML pages on Firebase Hosting, deployed from a private GitHub repository by GitHub Actions, with content edited directly in GitHub. Bookings run on Google Calendar appointment schedules, and Google Apps Script captures leads into a Google Sheet. There are no servers, no CMS and no backend code beyond one Apps Script project.

Stack at a glance

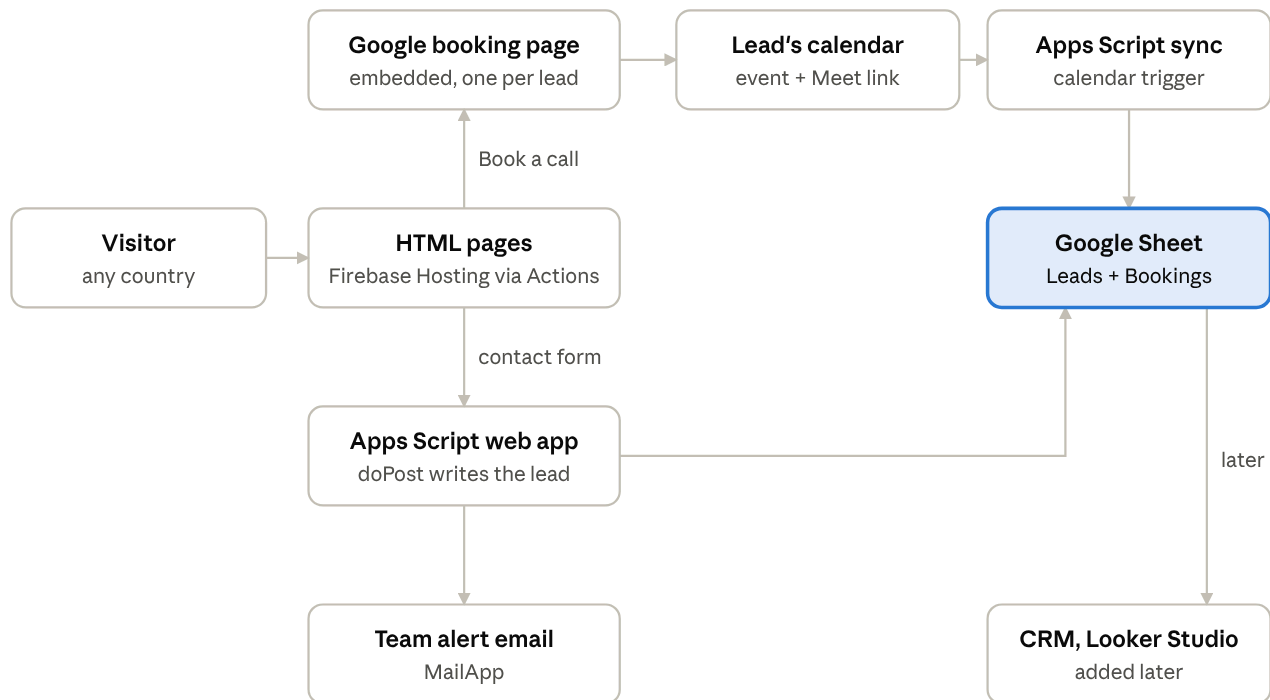
Five pieces, all free or already part of the team's Google Workspace and GitHub accounts.

Layer	Choice
Pages	Plain HTML, CSS and a little JavaScript, with Jekyll includes for the shared head, header and footer, built in GitHub Actions
Hosting	Firebase Hosting on sonso.co.in (global CDN, free SSL), deployed by GitHub Actions
Content editing	Directly in GitHub's web editor; every change is a commit with full history
Source of truth	One private GitHub repository: site, Apps Script code and docs
Bookings	Google Calendar appointment schedules, embedded on the site
Leads	HTML form, then a Google Apps Script web app, then a Google Sheet
Alerts and automations	Apps Script triggers and MailApp
Analytics	GA4, Meta Pixel and Google Ads tags in the shared head include

Architecture

Firebase Hosting serves the site and nothing else. Bookings go through Google's booking pages into each lead's calendar, and both bookings and contact-form leads end up in one Google Sheet.

Google tools handle bookings and leads; Firebase Hosting serves the site



lite architecture · pages, bookings, lead capture

The Sheet is the single register for every enquiry; the CRM sync and dashboard read from it when they are added.

Pages, repository and deployment

The site lives in a private GitHub repository. GitHub Actions builds it and deploys it to Firebase Hosting on every merge to `main`, and every pull request gets its own preview link.

```

sonso-site/
  .github/workflows/  firebase-hosting-merge.yml, firebase-hosting-pull-
request.yml
  firebase.json       hosting config: serves _site/, clean URLs
  .firebaserc         Firebase project ID
  _config.yml         Jekyll settings; excludes apps-script/, README.md and
Firebase files
  Gemfile             Jekyll version used by the build
  _includes/          head.html (meta tags, analytics), header.html,
footer.html
  _layouts/           default.html (wraps every page with the includes)
  index.html
  services/           build.html, automate.html, grow.html, measure.html
  work/               index.html, shehr-festival.html, gobig-
hospitality.html, marshal.html
  about.html, contact.html, book.html
  insights/           index.html + one file per article
  assets/             css/main.css, js/main.js, img/
  apps-script/        Code.gs (lead capture + booking sync)
  README.md           how to edit content safely

```

Each page starts with a three-line front matter block (`layout: default`, `title: ...`), so the shared head, navigation and footer are edited once and appear everywhere. Jekyll builds this inside GitHub Actions before each deploy.

Can GitHub Actions deploy straight to Firebase? Yes. Firebase maintains an official "Deploy to Firebase Hosting" GitHub Action, and it works with private repositories ([Firebase docs](#)). Running `firebase init hosting:github` once does the wiring: it creates a deploy service account, stores its key as a GitHub secret and writes the two workflow files.

Setup

1. Create a Firebase project (e.g. `sonso-site`).
2. Create the private repository in SoNSo's GitHub organisation.
3. Locally, run `firebase init hosting` with `_site` as the public directory, then `firebase init hosting:github` and follow the prompts.
4. Add the Jekyll build steps (below) to both generated workflow files, before the deploy step.
5. In the Firebase console, add `sonso.co.in` and `www.sonso.co.in` as custom domains, add the DNS records Firebase shows, and wait for the SSL certificate to be issued.

How deploys work

- **Pull request:** deploys to a preview channel and posts the preview URL as a comment on the PR.
- **Merge to `main`:** deploys to the live site.
- **Rollback:** Firebase Hosting keeps the release history; roll back to an earlier release from the console.

```
# .github/workflows/firebase-hosting-merge.yml (generated by the CLI, Jekyll
steps added)
name: Deploy to Firebase Hosting on merge
on:
  push:
    branches: [main]
jobs:
  build_and_deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: ruby/setup-ruby@v1
        with:
          ruby-version: '3.3'
          bundler-cache: true
      - run: bundle exec jekyll build    # outputs ./_site
      - uses: FirebaseExtended/action-hosting-deploy@v0
        with:
          repoToken: ${ secrets.GITHUB_TOKEN }
          firebaseServiceAccount: ${ secrets.FIREBASE_SERVICE_ACCOUNT_SONSO_SITE } # name as created by the CLI
          channelId: live
          projectId: sonso-site
# The pull-request workflow is the same without channelId: live, so it
deploys to a preview channel.
```

Editing content: open the page on github.com, click the pencil icon (or press `.` for the browser editor) and change the text. For a small fix, commit to `main` and it goes live in a few minutes. For anything larger, commit to a new branch and open a pull request to check the preview link first. Editable copy is marked with comments such as `<!-- EDIT: hero headline -->` so anyone can see what is safe to change.

Apps Script: the code in `apps-script/` is the source of truth and is pushed to Google with `clasp push`, or pasted into the Apps Script editor.

Bookings with Google Calendar appointment schedules

Each lead publishes a booking page from their own Google Workspace calendar, and the site embeds it. Google handles availability across time zones, double-booking prevention, Meet links, the calendar invite and the booking form.

Plan requirement. Business Starter accounts get a single booking page with the core features. Business Standard or higher adds multiple booking pages, automatic email reminders, checking availability across several calendars, verified bookings, Stripe payments and co-hosts ([Google: compare premium features](#)). Recommended: Business Standard for the three leads, mainly for reminders and verified bookings.

Setup for each lead

1. In Google Calendar choose Create, then Appointment schedule. Title it "Free 30-minute consultation", set 30 minutes, working hours in the lead's own time zone, a buffer between calls, minimum notice and a maximum bookings per day.
2. Add the co-host from the routing table below and tick their calendar under "Check calendars for availability". Turn on Google Meet, booking confirmations and email reminders.
3. Add booking form questions: company, website, what you need help with (Build / Automate / Grow / Measure), phone or WhatsApp (optional).
4. Under Sharing options, choose Website embed: an inline booking page for the Contact page, or a button with popup for the "Book a call" buttons ([Google: share your appointment schedule](#)).

Routing and co-hosts. Every consultation has two SoNSo people on it: the host who owns the booking page and a co-host.

Visitor needs	Host (booking page)	Co-host
Build	Akhil Gupta	Gattu Mani Sampath
Automate	Akhil Gupta	Gattu Mani Sampath
Grow	Nikhil Gupta	Gattu Mani Sampath
Measure	Gattu Mani Sampath	Nikhil Gupta
Not sure	Gattu Mani Sampath	Akhil Gupta

Co-hosts must be in the same Google Workspace domain, and Google does not check a co-host's calendar unless it is added to "Check calendars for availability" on the host's schedule ([Google: create an appointment schedule](#)). Each booking then lands on both people's calendars. Gattu Mani Sampath is on every meeting, so his calendar sets the overall availability, and he runs two schedules (Measure, Not sure), which needs Business Standard or higher. The "Book a call" page asks one question and shows the matching embedded booking page.

Lead capture: form to Apps Script to Google Sheet

One Google Sheet is the lead register. Contact-form leads arrive through an Apps Script web app, and bookings from the appointment schedules are copied in by an Apps Script calendar trigger, so every enquiry lands in one place.

Sheet structure

Tab	Columns
Leads	Timestamp · Name · Email · Company · Phone · Needs help with · Message · Page · UTM source · UTM campaign · Status · Owner
Bookings	Booked at · Meeting time (UTC) · Lead (host) · Name · Email · Form answers · Meet link · Calendar event ID · Status

Contact form → Leads tab

1. The HTML form posts form-encoded data with `fetch` to the Apps Script web app URL. A simple POST like this avoids CORS preflight issues.
2. `doPost` checks a hidden honeypot field, appends the row, and emails an alert to the team inbox.
3. The page shows a thank-you message with a "Book a call now" button.

```
// Code.gs: deploy as Web app, execute as Me, access: Anyone
const SHEET_ID = 'YOUR_SHEET_ID';

function doPost(e) {
  const p = e.parameter;
  if (p.website_url) return json({ ok: true }); // honeypot: bots fill it
  SpreadsheetApp.openById(SHEET_ID).getSheetByName('Leads').appendRow([
    new Date(), p.name, p.email, p.company, p.phone, p.stage,
    p.message, p.page, p.utm_source, p.utm_campaign, 'New', ''
  ]);
  MailApp.sendEmail('hello@sonso.co.in', 'New lead: ' + p.name,
    JSON.stringify(p, null, 2));
  return json({ ok: true });
}

function json(obj) {
  return ContentService.createTextOutput(JSON.stringify(obj))
    .setMimeType(ContentService.MimeType.JSON);
}
```

Bookings → Bookings tab

An installable Calendar trigger ("on calendar event updated") runs for each lead's calendar. Because the trigger does not say which event changed, the script uses the Calendar advanced service with a stored sync token to fetch only new or changed events. It keeps those that came from the booking page and appends the guest's name, email and form answers to the Bookings tab. A time-driven trigger every 15 minutes acts as a backstop. Each lead shares their calendar with the account that owns the script, or installs the trigger under their own account.

Automations to add later

Each of these is a few dozen lines of Apps Script on the same Sheet, added one at a time.

Automation	How
Auto-reply to contact-form leads with a booking link	MailApp in <code>doPost</code>
Assign an owner from "Needs help with"	Lookup in <code>doPost</code> , written to the Owner column

Automation	How
Daily digest of new leads and the day's bookings	Time-driven trigger, 9 a.m.
Push leads to the CRM (e.g. Zoho CRM)	UrlFetchApp call to the CRM API on each new row
WhatsApp confirmations	UrlFetchApp to the WhatsApp Business Cloud API
No-show and follow-up reminders	Daily trigger checks Bookings where the meeting has passed and Status is not set
Lead and booking dashboard	Looker Studio connected directly to the Sheet

Limits to know

- **Booking look and flow:** the booking page is Google's, shown in an iframe, so its styling and steps are only partly customisable.
- **Routing:** no round robin; routing is fixed per service, with a co-host on every schedule.
- **Apps Script quotas:** daily limits on emails sent and trigger run time apply. Fine at tens of leads a day.
- **Google Sheet as the register:** works well into the thousands of rows; archive old rows once a year.
- **Firebase Hosting:** serves static files here, with usage quotas on the no-cost tier ([Firebase Hosting usage, quotas and pricing](#)).
- **Public form endpoint:** the Apps Script web app URL is visible in the page source, so the honeypot and checks in `doPost` matter.

Open decisions

- ✓ ~~**Workspace plan:** Business Standard or higher for the three leads, required for co-hosts, reminders and Gattu Mani Sampath's two schedules.~~
- ✓ ~~**Hosts and co-hosts:** decided, as in the routing table (two people on every meeting).~~
- ✓ ~~**Repository and hosting:** private GitHub repository, deployed to Firebase Hosting by GitHub Actions.~~

☒ **Alert inbox:** ~~hello@sonso.co.in.~~

☐ **CRM:** later.

Sources: [Google Calendar Help, compare premium features for appointment schedules](#) · [Google Workspace Help, share your appointment schedule](#) · [Google Calendar Help, create an appointment schedule](#) · [Firebase, deploy to Hosting via GitHub](#)